



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sesión de Laboratorio 5:

BIT PERMUTATION-PART I



INTRODUCCIÓN A LA CRIPTOGRAFÍA

Profesora || Sandra Díaz Santiago

Grupo || 6CV4

Alumno || Areli Alejandra Guevara Badillo

Octubre 10, 2025

EJERCICIOS DE CALENTAMIENTO

1. Declara una variable data de tipo unsigned char, asígnale cualquier carácter ASCII imprimible, y usando printf imprime el valor como un carácter, hexadecimal y entero.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      unsigned char data;
6
7      printf("Ingrese un caracter: ");
8      scanf(" %c", &data);
9
10     printf("Caracter: '%c'\n", data);
11     printf("Hexadecimal: 0x%02X\n", data);
12     printf("Entero: %d\n\n", data);
13
14     return 0;
15 }
```

```
Ingrese un caracter: 5
Caracter: '5'
Hexadecimal: 0x35
Entero: 53
```

2. Utiliza el operador a nivel de bits << para hacer un desplazamiento a la izquierda de un solo bit, imprime el valor de data como un carácter, hexadecimal y entero. Repite ocho veces.

¿Qué puedes concluir al observar los resultados?

Cada desplazamiento a la izquierda multiplica por 2 el número original, pero como la variable solo tiene 8 bits, los bits 1 se pierden al llegar al final. Por esta razón, después de repetir el proceso 8 veces, todos los bits originales desaparecen y el valor final se convierte en 0.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      unsigned char data;
6
7      printf("Ingrese un caracter: ");
8      scanf(" %c", &data);
9
10     printf("Caracter: '%c'\n", data);
11     printf("Hexadecimal: 0x%02X\n", data);
12     printf("Entero: %d\n\n", data);
13
14     printf("Desplazamiento a la izquierda:\n");
15     unsigned char dataI = data;
16     for (int i = 0; i < 8; i++) {
17         dataI = dataI << 1;
18         printf("[%d]:\n", i + 1);
19         printf("Caracter: '%c'\n", dataI);
20         printf("Hexadecimal: 0x%02X\n", dataI);
21         printf("Entero: %d\n\n", dataI);
22     }
23
24     return 0;
25 }

```

```

Ingrese un caracter: 5
Caracter: '5'
Hexadecimal: 0x35
Entero: 53

Desplazamiento a la izquierda:
[1]:
Caracter: 'j'
Hexadecimal: 0x6A
Entero: 106

[2]:
Caracter: 'è'
Hexadecimal: 0xD4
Entero: 212

[3]:
Caracter: '¿'
Hexadecimal: 0xA8
Entero: 168

[4]:
Caracter: 'P'
Hexadecimal: 0x50
Entero: 80

[5]:
Caracter: 'á'
Hexadecimal: 0xA0
Entero: 160

[6]:
Caracter: '@'
Hexadecimal: 0x40
Entero: 64

[7]:
Caracter: 'Ç'
Hexadecimal: 0x80
Entero: 128

[8]:
Caracter: ''
Hexadecimal: 0x00
Entero: 0

```

3. Inicializa data con la letra 'A', usa el operador a nivel de bits >> para hacer un desplazamiento a la derecha de un solo bit, imprime el valor de data como un carácter, hexadecimal y entero. Repite ocho veces.

¿Qué puedes concluir al observar los resultados?

Cada desplazamiento a la derecha divide entre 2 el número original. Al repetir el proceso, el valor sigue disminuyendo hasta que, inevitablemente, se convierte en 0 una vez que todos los bits '1' han sido desplazados y eliminados.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      unsigned char data;
6
7      printf("Ingrese un caracter: ");
8      scanf(" %c", &data);
9
10     printf("Caracter: '%c'\n", data);
11     printf("Hexadecimal: 0x%02X\n", data);
12     printf("Entero: %d\n\n", data);
13
14     printf("Desplazamiento a la derecha:\n");
15     unsigned char dataD = data;
16     for (int i = 0; i < 8; i++) {
17         dataD = dataD >> 1;
18         printf("[%d]:\n", i + 1);
19         printf("Caracter: '%c'\n", dataD);
20         printf("Hexadecimal: 0x%02X\n", dataD);
21         printf("Entero: %d\n\n", dataD);
22     }
23
24     return 0;
25 }

```

```

Ingrese un caracter: A
Caracter: 'A'
Hexadecimal: 0x41
Entero: 65

Desplazamiento a la derecha:
[1]:
Caracter: ' '
Hexadecimal: 0x20
Entero: 32

[2]:
Caracter: ''
Hexadecimal: 0x10
Entero: 16

[3]:
Caracter: ' '
Hexadecimal: 0x08
Entero: 8

[4]:
Caracter: ''
Hexadecimal: 0x04
Entero: 4

[5]:
Caracter: ''
Hexadecimal: 0x02
Entero: 2

[6]:
Caracter: ''
Hexadecimal: 0x01
Entero: 1

[7]:
Caracter: ''
Hexadecimal: 0x00
Entero: 0

[8]:
Caracter: ''
Hexadecimal: 0x00
Entero: 0

```

Sesión de Laboratorio 5: Bit Permutation-Part I

- Declara dos variables `v1`, `v2`, ambas de tipo `unsigned char`, asígnales cualquier valor en binario, usa los operadores a nivel de bits AND (&), OR (|) y XOR (^), entre ellas. Imprime el resultado como un hexadecimal y entero.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     unsigned char v1, v2;
6
7     v1 = 0b0001111; // 15
8     v2 = 0b1111000; // 240
9
10    printf("v1 = 0001111 (Hexadecimal: 0x%02X, Entero: %d)\n", v1, v1);
11    printf("v2 = 1111000 (Hexadecimal: 0x%02X, Entero: %d)\n\n", v2, v2);
12
13    unsigned char and_result = v1 & v2;
14    unsigned char or_result = v1 | v2;
15    unsigned char xor_result = v1 ^ v2;
16
17    printf("v1 & v2:\nHexadecimal: 0x%02X\nEntero: %d\n\n", and_result, and_result);
18    printf("v1 | v2:\nHexadecimal: 0x%02X\nEntero: %d\n\n", or_result, or_result);
19    printf("v1 ^ v2:\nHexadecimal: 0x%02X\nEntero: %d\n\n", xor_result, xor_result);
20 }

```

```

v1 = 0001111 (Hexadecimal: 0x0F, Entero: 15)
v2 = 1111000 (Hexadecimal: 0xF0, Entero: 240)

v1 & v2:
Hexadecimal: 0x00
Entero: 0

v1 | v2:
Hexadecimal: 0xFF
Entero: 255

v1 ^ v2:
Hexadecimal: 0xFF
Entero: 255

```

Realiza cada operación a nivel de bits a mano con los valores dados, y compara los resultados que obtuviste con los que te dio tu programa.

v1 = 0 0 0 0 1 1 1 1																				
v2 = 1 1 1 1 0 0 0 0																				
AND (v1 & v2)																				
0 0 0 0 1 1 1 1				a	b	&	0 0 0 0 1 1 1 1				a	b		0 0 0 0 1 1 1 1				a	b	^
& 1 1 1 1 0 0 0 0				0	0	0	1 1 1 1 1 0 0 0 0				0	0	0	1 1 1 1 0 0 0 0				0	0	0
0 0 0 0 0 0 0 0				0	1	0	1 1 1 1 1 1 1 1				0	1	1	1 1 1 1 1 1 1 1				0	1	1
0x 00				1	0	0	0x FF				1	0	1	0x FF				1	0	1
0				1	1	1	255				1	1	1	255				1	1	0

5. Declara una variable `data2`, asígnale cualquier valor en binario. Haz una máscara de bits usando el operador a nivel de bits AND (&) entre `data2` y otro valor conveniente para obtener el bit más significativo de `data2`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     unsigned char data2 = 0b10101101; // 173
6     unsigned char mask = 0b10000000;
7
8     unsigned char msb = data2 & mask;
9
10    printf("data2 = 10101101\n");
11    printf("Mascara = 10000000\n");
12
13    printf("Bit mas significativo: %d\n", msb ? 1 : 0);
14    // valor ≠ 0 → 1, valor = 0 → 0
15 }
```

```
data2 = 10101101
Mascara = 10000000
Bit mas significativo: 1
```

Realiza esta operación a mano y compara tu resultado con el resultado de tu programa.

data2 = 1 0 1 0 1 1 0 1

mask = 1 0 0 0 0 0 0 0

AND (v1 & v2)

	1	0	1	0	1	1	0	1
&	1	0	0	0	0	0	0	0
	1	0	0	0	0	0	0	0

a	b	&
0	0	0
0	1	0
1	0	0
1	1	1

Cuando es diferente de 0 el resultado (128), nos da 1, que es el bit mas significativo de data2, en caso contrario el bit más significativo es 0.

6. Usa nuevamente la variable data2, asígnale cualquier valor en binario. Haz una máscara de bits usando el operador a nivel de bits AND (&) entre data2 y otro valor conveniente para obtener los cuatro bits menos significativos de data2.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     unsigned char data2 = 0b10101101; // 173
6     mask = 0b00001111;
7     unsigned char lsb = data2 & mask;
8
9     printf("data2 = 10101101\n");
10    printf("Mascara = 00001111\n");
11    printf("Cuatro bits menos significativos: 0x%02X\n", lsb);
12 }

```

```

data2 = 10101101
Mascara = 00001111
Cuatro bits menos significativos: 0x0D

```

Realiza esta operación a mano y compara tu resultado con el resultado de tu programa.

data2 = 1 0 1 0 1 1 0 1							
mask = 0 0 0 0 1 1 1 1							
AND (v1 & v2)							
1	0	1	0	1	1	0	1
&	0	0	0	0	1	1	1
0	0	0	0	1	1	0	1
0x 0D							
13							

a	b	&
0	0	0
0	1	0
1	0	0
1	1	1

EJERCICIOS DE PROGRAMACIÓN

1. Escribe un programa para obtener el n-ésimo bit de un número.

Obtener el n-ésimo bit de un número

```
1 unsigned char bitGet(unsigned char num, int n) {
2     return (num >> n) & 1;
3 }
```

Funcionamiento:

	7	6	5	4	3	2	1	0
num =	0	0	0	1	1	0	0	1
Consultar el bit en la posición 5: 0								
Se desplaza 5 veces a la derecha:								
[0]	0	0	0	1	1	0	0	
[1]	0	0	0	0	1	1	0	0
[2]	0	0	0	0	0	1	1	0
[3]	0	0	0	0	0	0	1	1
[4]	0	0	0	0	0	0	0	1
[5]	0	0	0	0	0	0	0	0

⇒

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

0x00
0

Ingresa el numero: 25
Bits de 25: 00011001

Posicion de bit a consultar: 5
El bit 5 de 25 es: 0

2. Escribe un programa para establecer el n-ésimo bit de un número.

Establecer el n-ésimo bit de un número

```
1 unsigned char bitSet(unsigned char num, int n) {
2     return num | (1 << n);
3 }
```

Funcionamiento:

	7	6	5	4	3	2	1	0
num =	0	0	0	1	1	0	0	1
Establecer en 1 el bit en la posición: 7								
Se desplaza 7 veces a la izquierda:								
[0]	0	0	0	0	0	0	0	1
[1]	0	0	0	0	0	0	1	0
[2]	0	0	0	0	0	1	0	0
[3]	0	0	0	0	1	0	0	0
[4]	0	0	0	1	0	0	0	0
[5]	0	0	1	0	0	0	0	0
[6]	0	1	0	0	0	0	0	0
[7]	1	0	0	0	0	0	0	0

⇒

0	0	0	1	1	0	0	1
1	1	0	0	0	0	0	0
1	0	0	1	1	0	0	1
0	x	9	9				
1	5	3					

0x99
153

Ingresa el numero: 25
Bits de 25: 00011001

Posicion de bit a establecer a 1: 7
Estableciendo el bit 7: 10011001 (Valor: 153)

3. Escribe un programa para contar los ceros iniciales en un número binario.

Contar los ceros iniciales en un número binario

```
1 int cerosIni(unsigned char num) {
2     int c = 0;
3     for (int i = 7; i ≥ 0; i--) {
4         if ((num >> i) & 1)
5             break;
6         c++;
7     }
8     return c;
9 }
```

Funcionamiento:

No. de ceros iniciales: 3												
		7	6	5	4	3	2	1	0			
num =		0	0	0	1	1	0	0	1			
c	i	num >> i							&1	¿es 1?	c	return 3
0	7	0	0	0	0	0	0	0	0	×	1	
	6	0	0	0	0	0	0	0	0	×	2	
	5	0	0	0	0	0	0	0	0	×	3	
	4	0	0	0	0	0	0	1	1	✓		

Ingresa el numero: 25
 Bits de 25: 00011001
 Ceros iniciales en 25: 3